TREINAMENTO TÉCNICO / PT-BR

DryRun Security

Do primeiro scan à remediação

Configuração, execução e diagnóstico
com laboratórios e comandos reproduzíveis

---

01 **GETTING STARTED**
Documentação e primeiro repositório

02 **SCANNING**
PR, DeepScan, SCA, Secrets e IaC

03 **PLATFORM**
Risco, contexto, políticas, MCP e API

04 **API INTEGRATIONS**
Webhooks, Jira, chaves e remediação

25 MÓDULOS • 25 LABORATÓRIOS • KIT DE SCRIPTS

Edição 20/10/2026 | Material independente de treinamento

Sumário

Como usar este treinamento 3
Preparação do ambiente e convenções 4
01. Documentation 6
02. Quick Start 7
03. PR Scanning 9
04. SCA - Dependencies 10
05. Repository Scanning with DeepScan 11
06. Secrets Scanning 12
07. IaC Scanning 13
08. Language and Framework Support 14
09. Vulnerability Coverage Matrix 15
10. Reporting 16
11. Risk Register 17
12. Code Security Intelligence 18
13. PR Scanning Configuration 19
14. Custom Code Policies 20
15. Repository Context 21
16. Finding Tuning with Feedback 22
17. PR Blocking 23
18. Permissions 25
19. MCP 26
20. DryRun API 27
21. Webhooks 29
22. Jira Integration 31
23. API Access Keys 33
24. Remediation 34
25. DryRun Skill 35
Exercício final - do risco à evidência 36
Referência rápida de diagnóstico 37
Apêndice A - arquivos completos do kit 38
Apêndice B - fontes e rastreabilidade 42

Como usar este treinamento

Este curso acompanha o ciclo completo: preparar um repositório, provocar e corrigir um finding em uma PR, investigar o repositório com DeepScan, organizar o risco, ajustar políticas e integrar evidências a ferramentas externas. Cada tópico solicitado corresponde a um módulo. O item “DryRun” da seção Platform foi interpretado como “DryRun API”, conforme o PDF fornecido.

O percurso sugerido é de 24 a 32 horas, distribuídas em quatro encontros. A duração é uma estimativa didática, não um SLA de execução da plataforma. Os módulos 1 a 9 cobrem instalação e scanners; 10 a 18, operação e governança; 19 e 20, acesso programático; 21 a 25, integrações e remediação. A conclusão de scans depende da fila, do tamanho do repositório e dos recursos da conta.

Em cada módulo: compreenda o mecanismo; acompanhe um caso de uso; execute o laboratório; registre as evidências; aplique o roteiro de diagnóstico. O laboratório termina com validação e limpeza. As funcionalidades que só têm configuração documentada na interface são descritas como passos de interface. Não existe um comando inventado “dryrun scan” neste material.

Os blocos de terminal usam Bash, macOS ou Linux. No Windows, use WSL2 com Ubuntu. O kit ZIP contém os arquivos transcritos no apêndice: ele reduz erros de cópia, mas o PDF permanece autocontido. Funções como lab_branch, lab_pr e dryget pertencem ao curso; não são CLIs fornecidas pelo DryRun.

Use um repositório privado dedicado. Os exercícios introduzem falhas de propósito e não precisam de deploy público. A aplicação HTTP fica em 127.0.0.1. Não mescle versões vulneráveis na main. Apenas o receptor de webhook do módulo 21 usa uma URL pública temporária, protegida por um caminho aleatório para o laboratório.

Uma ausência de finding não prova ausência de vulnerabilidade. O resultado depende do contexto, do diff, da configuração e do comportamento da análise. As expectativas do curso distinguem evidência funcional, evidência do scanner e inferência humana.

Preparação do ambiente e convenções

Acessos e ferramentas

Você já tem DryRun e GitHub. Confirme que consegue instalar/autorizar a GitHub App para o repositório de teste. Administração de configurações, chaves, DeepScan e políticas pode exigir papel Admin. Os módulos Jira exigem adicionalmente um projeto Jira Cloud, permissão para criar issues e um token de API. O módulo Codex exige uma instalação e autenticação válidas do Codex. Esses acessos não são presumidos a partir do acesso ao GitHub.

Instale as ferramentas pela opção do seu sistema. Os comandos abaixo pressupõem Homebrew já instalado no macOS, ou privilégios sudo no Ubuntu. Node/npm só são necessários para Codex e DryRun Skill; cloudflared só no laboratório de webhook.

```
# macOS
brew install git gh jq python node

# Ubuntu / WSL2
sudo apt-get update
sudo apt-get install -y git gh jq python3 curl unzip nodejs npm

# Ambos: confira as versoes efetivamente instaladas
bash --version
git --version
gh --version
jq --version
python3 --version
curl --version
node --version
npm --version
```

Se os pacotes da distribuição forem antigos demais para a CLI usada, atualize pelo distribuidor oficial antes do módulo correspondente. Não é necessário instalar pacotes Python para a aplicação: ela usa apenas a biblioteca padrão. As dependências vulneráveis do laboratório SCA ficam em manifesto, sem instalação.

Workspace, kit e evidências

Abra Bash e mantenha a sessão durante os módulos. Substitua o caminho do ZIP pelo caminho em que você o salvou. Alternativamente, transcreva os arquivos do apêndice em uma pasta kit e defina KIT para ela. Não coloque o kit inteiro no repositório: isso misturaria exemplos vulneráveis, de webhook e de remediação com o alvo de análise.

```
bash
mkdir -p "$HOME/dryrun-training"
cd "$HOME/dryrun-training"
unzip /CAMINHO/kit-treinamento-dryrun.zip -d kit
export KIT="$PWD/kit"
export LAB_ROOT="$PWD"
source "$KIT/lab.sh"
mkdir -p "$LAB_ROOT/repo"
cd "$LAB_ROOT/repo"
```

Use .evidence para dados de execução. Essa pasta será ignorada pelo Git. Não publique tokens, respostas de API com dados sensíveis ou payloads de clientes. Quando um terminal novo for aberto, redefina KIT, LAB_ROOT e GH_REPO, carregue lab.sh e entre em repo. As variáveis AID, RID, SID, FID e PR são IDs obtidos em respostas reais, nunca os nomes de exibição.

Preparação antecipada da API

Para os laboratórios que consultam dados antes do módulo de chaves, um administrador deve abrir Settings > Access Keys, criar uma chave temporária e selecionar as permissões de leitura necessárias. O módulo 23 explica o ciclo completo. A chave é exibida uma única vez. Copie-a para um gerenciador seguro e carregue apenas na sessão. Não há endpoint de emissão de chave usado neste curso.

```
read -rsp 'DryRun API key: ' DRYRUN_API_KEY; echo
export DRYRUN_API_KEY
dryget accounts | tee /tmp/dryrun-accounts.json | jq .
# Escolha o account_id correto da resposta, sem assumir o primeiro
read -rp 'Account ID: ' AID
export AID
```

Caso ainda não possa criar uma chave, conclua os módulos de interface e retorne aos exercícios de API após o módulo 23. Para o POST de DeepScan será necessário escopo de escrita apropriado, além do acesso ao repositório. Não amplie escopos de uma chave de leitura só para contornar um 403 sem confirmar o papel necessário.

GETTING STARTED

01. Documentation

Mecanismo e decisões técnicas

DryRun usa análise contextual de código para relacionar uma alteração às funções, fluxos e controles de segurança que a cercam. A unidade de trabalho muda conforme o recurso: PR Scanning analisa mudanças propostas; DeepScan explora um repositório em uma referência; SCA examina dependências; o Risk Register consolida achados para tratamento. Confundir essas unidades leva a interpretar um scan de PR limpo como avaliação completa de uma aplicação.

O fluxo operacional começa no SCM, onde o evento da PR inicia a análise. Os resultados alimentam o comentário e os checks, o painel e as integrações habilitadas. Um status check reporta um resultado; uma regra de proteção do GitHub transforma esse resultado em impedimento de merge. O painel permite investigação e triagem; as APIs e o MCP expõem contextos diferentes para automação e perguntas.

LLMs participam da interpretação contextual e de recursos como Code Security Intelligence. A configuração, o código e os documentos de contexto ajudam a formular essa interpretação. MCP é o protocolo de integração entre um cliente de IA e ferramentas expostas pelo servidor; não é um scanner nem uma substituição da proteção de branch. Uma resposta do assistente precisa ser rastreável às evidências retornadas pelas ferramentas.

Caso de uso

Uma equipe deseja saber se um patch em autorização cria um IDOR. O fluxo correto é examinar o diff e a rota no PR Scanning, entender o modelo de autorização com Repository Context, conferir alcance e variantes com DeepScan e registrar a decisão de tratamento. Um resumo do LLM sem caminhos, linhas ou finding IDs não basta para aprovar a mudança.

Laboratório 01 - inventário de acesso

Objetivo: estabelecer a identidade do executor e o escopo do treinamento. Execute a autenticação do GitHub e salve um inventário sem credenciais.

```
gh auth login
gh auth setup-git
gh auth status
gh api user --jq '{login, id}'
mkdir -p .evidence
cat > .evidence/plano.md <<'TXT'
Ativo: repositorio privado de treinamento
Ambiente: local, sem dados reais
Objetivo: PR -> triagem -> correcao -> evidencias
Fontes: PDFs DryRun fornecidos em 20/10/2026
TXT
```

Na interface DryRun, confira a organização selecionada e os repositórios visíveis. Registre seu papel e a organização em plano.md. Critério: identidade e organização correspondem ao GitHub alvo. Diagnóstico: repositório ausente pode ser falta de instalação/seleção na GitHub App, organização incorreta ou permissão do usuário. Limpeza: nenhuma mudança de aplicação foi feita; mantenha o inventário.

Fonte: Documentation, páginas 1-2; Quick Start, páginas 1-7.

GETTING STARTED

02. Quick Start

Mecanismo e decisões técnicas

A GitHub App precisa ter acesso ao repositório antes de receber eventos relevantes. Permitir uma organização não implica necessariamente selecionar todos os repositórios: instalações limitadas a repositórios específicos precisam incluir o novo laboratório. O quick start deve terminar com uma PR rastreável, não apenas com o aplicativo instalado.

Uma base inicial com código seguro permite comparar o comportamento antes e depois de cada alteração. A main conterá a aplicação mínima e dois testes: consulta normal e payload tratado como valor literal. Cada laboratório começa de main, abre uma branch própria e fecha a PR sem merge quando houver código vulnerável. Isso evita que o exercício seguinte herde uma falha anterior.

Caso de uso

Você quer demonstrar DryRun em uma conta que já tem vários projetos. Um repositório novo permite aplicar configurações e checks sem alterar regras de produção. Ele também evita ambiguidade com o remote SSH github-veracode e com a branch local main inexistente do erro de push observado anteriormente.

Laboratório 02 - criar a base

Escolha um nome disponível. GH_OWNER pode ser sua conta ou uma organização na qual você tenha permissão de criação. Se a organização impedir repositórios privados desse tipo, solicite um repositório de treinamento ao administrador e clone-o em vez de executar gh repo create.

```
read -rp 'Conta ou organizacao GitHub: ' GH_OWNER
export GH_REPO="$GH_OWNER/dryrun-training"
git init -b main
if ! git config user.name >/dev/null; then
  read -rp 'Seu nome para os commits: ' COMMIT_NAME
  git config --local user.name "$COMMIT_NAME"
fi
if ! git config user.email >/dev/null; then
  read -rp 'Seu email para os commits: ' COMMIT_EMAIL
  git config --local user.email "$COMMIT_EMAIL"
fi
cp "$KIT/app.py" "$KIT/test_app.py" .
cat > .gitignore <<'TXT'
.evidence/
__pycache__/
.venv/
.env
TXT
printf '# DryRun Training\n\nAmbiente didatico privado.\n' > README.md
python3 -m unittest -v
git add app.py test_app.py .gitignore README.md
git commit -m 'Base segura do treinamento'
gh repo create "$GH_REPO" --private --source . --remote origin --push
```

No DryRun, siga o onboarding da organização e autorize a GitHub App para esse repositório. Verifique no GitHub Settings > Integrations > GitHub Apps a instalação correspondente. Não há comando universal para conceder consentimento de instalação em nome do administrador.

```
lab_branch lab/02-onboarding
printf '\nPrimeira PR de validacao.\n' >> README.md
git add README.md
git commit -m 'Valida integracao com PR'
lab_pr 'Lab 02 - onboarding'
lab_evidence
```

Critério: existe evidência de análise/check no SHA da PR ou registro correspondente no DryRun. Uma PR apenas documental pode não produzir achados; ausência de comentário é esperada em execução limpa, dependendo da configuração. Não conclua que houve scan só porque não há comentário. Confira os checks. Ao concluir, execute lab_close.

Diagnóstico do push: src refspec main does not match any significa que a referência local não existe, frequentemente por falta do primeiro commit ou por a branch ter outro nome. Execute os comandos de inspeção abaixo no repositório afetado antes de alterar qualquer branch.

```
git status
git branch --show-current
git log -1 --oneline
git remote -v
# Repo novo sem commits: adicione arquivos e faca o primeiro commit.
# Se a branch atual deve realmente ser main:
git branch -M main
# Use o remote correto; no seu exemplo anterior, ele era dryrun:
# git push -u dryrun main
```

Não copie o texto HTML   para o terminal. Se main já existir localmente com outro trabalho, não use branch -M para sobrescrevê-la: compare as branches primeiro.

Fonte: Quick Start, páginas 1-7.

SCANNING

03. PR Scanning

Mecanismo e decisões técnicas

O scan de PR avalia o diff e consulta o contexto necessário para entender seu efeito. Abertura e atualizações da PR geram novas análises. O código legado fora da alteração não deve ser tratado como integralmente revisado. Drafts podem ser analisados, mas os comentários aguardam a PR ficar pronta para revisão.

Por padrão, uma análise sem findings não produz comentário. Se os findings forem corrigidos em um commit seguinte, o comentário pode ser removido. A opção Show comment for no findings altera isso para manter uma confirmação explícita. Checks e registro de scan são evidência mais confiável do que presença ou ausência de comentário.

O laboratório usa uma SQL injection em SQLite: concatenar a entrada ao SQL muda a estrutura da consulta. Parametrizar preserva a entrada como dado. O teste local demonstra essa diferença mesmo que o scanner não produza o finding esperado. A análise automatizada e a prova funcional são evidências complementares.

Caso de uso

Um desenvolvedor substitui uma consulta parametrizada por concatenação para “simplificar” um filtro. O fluxo de dados vem da query string HTTP, chega à função search e termina em execute. Esse encadeamento permite avaliar controlabilidade da entrada e impacto da operação.

Laboratório 03 - introduzir e corrigir SQLi

```
lab_branch lab/03-sqli
python3 "$KIT/change.py" unsafe
python3 -m unittest -v
# O teste de injecao DEVE falhar nesta etapa.
git diff -- app.py
git add app.py
git commit -m 'Lab: consulta insegura para avaliacao'
lab_pr 'Lab 03 - detectar SQL injection'
lab_evidence
# Acompanhe os checks; interrompa com Ctrl+C se o scan nao iniciar.
gh pr checks --watch --interval 15
```

Abra a PR e o finding no DryRun. Registre arquivo, linha, origem da entrada, sink, risco e recomendação. Um check de falha é um resultado esperado se blocking estiver ativo. O comando gh pr checks pode retornar código diferente de zero por findings ou por checks pendentes; isso não significa falha de autenticação.

```
python3 "$KIT/change.py" safe
python3 -m unittest -v
git add app.py
git commit -m 'Corrige SQLi com parametro vinculado'
git push
lab_evidence
```

Critério: os dois testes passam e o novo scan avalia o SHA corrigido. Se o finding continuar, confira se o scan terminou, se a recomendação se refere à mesma linha e se existe outro caminho vulnerável. Se nada for detectado na versão vulnerável, confira diff, agentes, escopo e detalhes do scan; não altere severidade apenas para obter a demonstração. Limpeza: lab_close.

Fonte: PR Scanning, páginas 1-5; PR Scanning Configuration, páginas 3-4.

SCANNING

04. SCA - Dependencies

Mecanismo e decisões técnicas

SCA identifica componentes e vulnerabilidades conhecidas a partir dos manifests e lockfiles reconhecidos. A versão resolvida é importante: um intervalo no manifesto não comprova a versão efetivamente instalada. O lockfile ajuda a reconstruir a árvore de dependências e sua relação com o projeto.

A documentação descreve SCA dentro de DeepScan e scans em segundo plano. A frequência destes depende da atividade de PRs nos últimos 30 dias; repositórios com menos de três PRs não entram nesse agendamento. Por isso, não espere que um laboratório novo receba imediatamente o scan de fundo. Use DeepScan para executar o percurso de forma controlada.

Em Gradle, build.gradle isolado não basta para resolver a lista de componentes: utilize um lockfile suportado e commitado. A documentação também descreve verification-metadata.xml na raiz para o caso suportado. SBOM é inventário de componentes; ele não é uma lista equivalente de vulnerabilidades. Ausência de CVE associada não torna automaticamente um componente confiável.

Caso de uso

A equipe precisa comprovar quais versões foram avaliadas antes de atualizar uma biblioteca. Exporta o SBOM, identifica dependências diretas e transitivas, decide a atualização e executa testes de compatibilidade. A decisão não deve usar apenas a contagem de CVEs.

Laboratório 04 - manifesto e inventário

O pin antigo abaixo é uma semente didática; não instale nem execute essa dependência. A correspondência com advisories será determinada pelo scanner e pela base consultada, sem promessa de um CVE específico.

```
lab_branch lab/04-sca
printf 'requests==2.19.1\n' > requirements.txt
git add requirements.txt
git commit -m 'Lab: manifesto para SCA'
lab_pr 'Lab 04 - inventario de dependencias'
```

Na interface DeepScan, selecione o repositório e a branch lab/04-sca e inicie o scan. Ao concluir, abra a seção de dependências e exporte SBOM no formato disponível. O módulo 5 explica o DeepScan e o módulo 20 mostra os endpoints; você pode consultá-los agora. O POST de API demonstrado no curso não especifica uma branch não documentada.

```
# Depois de definir AID, RID e SID reais nos modulos 5/20:
dryget "accounts/$AID/repositories/$RID/deepscans/$SID/sca_results" \
  > .evidence/sca.json
dryget "accounts/$AID/repositories/$RID/deepscans/$SID/sbom" \
  > .evidence/sbom.json
jq 'type' .evidence/sca.json .evidence/sbom.json
```

Critério: localizar requests e sua versão no inventário, ou registrar claramente por que o manifesto não foi processado. Para uma correção real, escolha uma versão suportada após revisar advisories e compatibilidade; gere novamente o lockfile no projeto real. Diagnóstico: confira branch, arquivo commitado, formato suportado, término da etapa SCA e retorno HTTP. Limpeza: lab_close; não mescle o pin antigo.

Fonte: SCA, páginas 1-6; DryRun API, páginas 1-6.

SCANNING

05. Repository Scanning with DeepScan

Mecanismo e decisões técnicas

DeepScan é a avaliação de repositório inteiro, inclusive código não alterado recentemente. A sequência documentada inclui perfil do projeto, mapeamento de rotas e autenticação, planejamento por domínios de segurança, investigação, consolidação/calibração e relatório. Isso o torna adequado para baseline, aquisição de uma aplicação ou revisão de superfícies legadas.

O histórico Git fornece sinais para orientar investigação: frequência de mudanças, autores, recência, complexidade e arquivos que costumam mudar juntos. Esses sinais ajudam a priorizar atenção, mas não são prova de vulnerabilidade nem avaliação individual de desempenho de um desenvolvedor.

O scan pode ser iniciado para uma branch pela interface. Registre a referência e o commit analisados: um relatório antigo não valida o estado atual. Contexto específico configurado para DeepScan substitui o conteúdo de AGENTS.md para essa análise, enquanto, na ausência desse contexto, DeepScan pode descobrir arquivos AGENTS.md inclusive em subdiretórios.

Caso de uso

Uma API com poucos PRs recentes tem um controle de autorização vulnerável há meses. PR Scanning não deve ser usado como auditoria desse legado. DeepScan fornece uma avaliação mais ampla, que depois é convertida em remediações pontuais com PRs verificáveis.

Laboratório 05 - baseline e seleção de IDs

Use main como baseline seguro ou a branch do módulo 4 para incluir o manifesto SCA. No painel, selecione repositório/branch e inicie DeepScan. Observe as fases sem interpretar estados intermediários como resultado final.

```
git rev-parse main
mkdir -p .evidence
dryget "accounts/$AID/repositories" \
  | tee .evidence/repositories.json | jq .
read -rp 'Repository ID retornado pela API: ' RID
export RID
dryget "accounts/$AID/repositories/$RID/deepscans" \
  | tee .evidence/deepscans.json | jq .
read -rp 'ID do DeepScan concluido: ' SID
export SID
dryget "accounts/$AID/repositories/$RID/deepscans/$SID/results" \
  > .evidence/deepscan-results.json
jq . .evidence/deepscan-results.json
```

Critério: relacionar repositório, branch, commit, execução e resultados; identificar no relatório ao menos as superfícies avaliadas ou a limitação encontrada. Diagnóstico: fila/processing requer aguardar; 404 pode ser ID de scan de PR usado como DeepScan, conta incorreta ou falta de acesso; lista vazia pode significar que o scan não foi iniciado nessa conta. Limpeza: mantenha o relatório privado e feche a PR do módulo 4 caso ainda esteja aberta.

Fonte: Repository Scanning with DeepScan, páginas 1-5.

SCANNING

06. Secrets Scanning

Mecanismo e decisões técnicas

O scanner de segredos procura credenciais no código e usa contexto para diferenciar material suspeito de exemplos e valores inofensivos. A documentação menciona tentativas de validação de atividade quando aplicável. Uma detecção não substitui a decisão operacional sobre revogação; um segredo real já commitado deve ser tratado como potencialmente exposto mesmo após remoção do arquivo.

Uma credencial fictícia pode não ser reportada por ter formato inválido ou contexto de teste. Para observar comportamento sem expor uma conta, o laboratório gera uma chave privada descartável que nunca será instalada, confiada ou usada para autenticação. O finding continua não sendo garantido: o exercício também avalia como documentar uma não detecção.

Caso de uso

Uma chave real é enviada por engano em uma PR. A ordem operacional é revogar/rotacionar no provedor, atualizar consumidores, confirmar funcionamento e só então tratar arquivos e histórico conforme o processo da equipe. Apagar a linha do diff não invalida cópias existentes da credencial.

Laboratório 06 - material criptográfico descartável

```
lab_branch lab/06-secret
mkdir -p fixtures
ssh-keygen -t rsa -b 2048 -N '' \
  -C 'dryrun-lab-never-trusted' -f fixtures/lab_only_key
# Nunca adicione a chave publica a contas, servidores ou authorized_keys.
git add fixtures/lab_only_key fixtures/lab_only_key.pub
git commit -m 'Lab: chave descartavel nunca utilizada'
lab_pr 'Lab 06 - secrets scanning'
lab_evidence
```

Confira se o finding distingue uma chave privada do arquivo público. Registre categoria, severidade e eventual informação de validação sem copiar todo o segredo para um ticket. Em um incidente real, o comando de revogação depende do provedor e do tipo de credencial; não existe revogação universal pelo DryRun.

```
git rm fixtures/lab_only_key fixtures/lab_only_key.pub
git commit -m 'Remove material do laboratorio'
git push
lab_evidence
lab_close
```

Critério: comparar scans antes/depois e explicar que a chave continua no histórico da branch, apesar da remoção do arquivo. Diagnóstico: comentários desativados, agente silencioso, diff ou formato não reconhecido podem explicar ausência de comentário. Limpeza: feche a PR; a chave permanece sem confiança e sem utilidade fora da amostra. Não reutilize esse material em nenhum serviço.

Fonte: Secrets Scanning, páginas 1-2.

SCANNING

07. IaC Scanning

Mecanismo e decisões técnicas

A documentação destaca Terraform (.tf) em PRs e riscos como permissões excessivas, exposição de rede, defaults inseguros e situações relacionadas a subdomain takeover. A análise é estática: não é necessário executar terraform apply nem criar infraestrutura para avaliar o código.

O contexto muda o significado de uma configuração: uma regra ingress para toda a Internet em uma porta administrativa merece investigação diferente de uma porta pública de serviço. Da mesma forma, uma política IAM ampla pode ser exigida por um fluxo específico, mas deve ter justificativa verificável e limites apropriados.

Caso de uso

Uma mudança libera SSH para 0.0.0.0/0. O review deve identificar a superfície criada, verificar a necessidade operacional e limitar a origem à rede de administração. O exercício compara uma abertura ampla com uma faixa de documentação, sem implantar recursos reais.

Laboratório 07 - exposição em Terraform

```
lab_branch lab/07-iac
mkdir -p infra
cat > infra/security.tf <<'TF'
resource "aws_security_group" "lab" {
  name = "dryrun-training"
  ingress {
    from_port   = 22
    to_port     = 22
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }
}
TF
git add infra/security.tf
git commit -m 'Lab: regra de rede ampla'
lab_pr 'Lab 07 - Terraform exposto'
lab_evidence
```

Examine o finding e seu raciocínio. Faça a alteração abaixo; 192.0.2.0/24 é somente uma faixa de exemplo para documentação. Em produção, substitua por uma origem administrativa realmente autorizada e revise a arquitetura de acesso.

```
python3 - <<'PY'
from pathlib import Path
p = Path('infra/security.tf')
p.write_text(p.read_text().replace('0.0.0.0/0', '192.0.2.0/24'))
PY
git add infra/security.tf
git commit -m 'Lab: restringe origem administrativa'
git push
lab_evidence
lab_close
```

Critério: registrar a diferença entre regra ampla e restrita e o resultado de cada scan. Não afirmar que o segundo arquivo é uma infraestrutura completa ou pronta para deploy. Diagnóstico: confirme extensão, diff e suporte do scanner; um arquivo pequeno sem contexto pode gerar um resultado diferente de um módulo Terraform real. Limpeza: PR fechada, nenhum recurso cloud criado.

Fonte: IaC Scanning, páginas 1-2.

SCANNING

08. Language and Framework Support

Mecanismo e decisões técnicas

A lista documentada inclui JavaScript/TypeScript, Python, Java, Go, Ruby, PHP, C#, Kotlin, Swift e Rust. O reconhecimento de linguagem e framework orienta a investigação sem exigir a mesma configuração de compilação de uma ferramenta baseada exclusivamente em build. Isso não implica cobertura idêntica de todos os frameworks, DSLs, geradores e bibliotecas de cada ecossistema.

Uma avaliação séria de cobertura verifica quatro elementos: linguagem reconhecida; entrada identificada; propagação compreendida; operação sensível contextualizada. Em um framework proprietário, Repository Context pode explicar wrappers de autenticação e rotas, mas não transforma automaticamente qualquer fluxo em análise garantida.

Caso de uso

Um monorepo possui serviço Python, frontend TypeScript e módulos Java. A equipe precisa validar caminhos representativos de cada stack antes de concluir que o produto cobre todo o conjunto. Um finding em Python não é evidência de entendimento do middleware Java.

Laboratório 08 - inventário e amostragem

```
git switch main
mkdir -p .evidence
git ls-files | python3 -c '
import sys, pathlib, collections
c = collections.Counter(pathlib.Path(x.strip()).suffix or "[sem extensao]"
                        for x in sys.stdin)
for ext, n in sorted(c.items()): print(ext, n)
' | tee .evidence/languages.txt
gh api "repos/$GH_REPO/languages" | tee .evidence/github-languages.json
```

Abra o perfil do DeepScan e compare com o inventário. GitHub Languages mede composição por bytes e pode tratar arquivos gerados de forma própria; não é o detector interno do DryRun. Para o repo didático, identifique Python e o uso de http.server/sqlite3. Em um projeto adicional autorizado, repita o inventário e escolha uma rota real por stack para revisar.

```
cat > .evidence/coverage-stack.md <<'TXT'
Stack: Python / biblioteca padrao
Entrada: GET /search?q=
Propagacao: Handler.do_GET -> search
Sink: sqlite3.execute
Controle: consulta parametrizada
Evidencia DryRun: preencher scan ID e observacao
Limite: amostra nao representa todos os frameworks
TXT
```

Critério: produzir um mapa de cobertura com evidência por stack e registrar o que não foi testado. Diagnóstico: arquivos ignorados, branch divergente e código gerado podem explicar diferenças. Limpeza: não há alteração a publicar; mantenha o inventário em .evidence.

Fonte: Language and Framework Support, páginas 1-2.

SCANNING

09. Vulnerability Coverage Matrix

Mecanismo e decisões técnicas

A matriz documenta classes de vulnerabilidade, descrições e CWEs associados. Ela orienta um plano de avaliação, mas não promete detectar toda ocorrência de cada CWE em qualquer contexto. Algumas classes dependem de entender autorização de negócio; outras dependem de fluxo de dados, uso criptográfico ou configuração.

Organize a leitura em famílias: injeções e interpretação de entrada; autenticação/autorização; exposição de dados e segredos; SSRF e acesso a recursos; criptografia e configuração; lógica de negócio e uso de LLM. Entre as classes documentadas está LLM Tool Misuse and Prompt Injection, com CWE-1426 e CWE-1427. Essa cobertura deve ser avaliada com o código e os limites de confiança da integração, não apenas com o texto de um prompt.

Para modelagem de ameaça, use a matriz como lista de hipóteses. Descreva ativo, ator, entrada, fronteira de confiança, caminho de abuso e controle esperado; depois procure evidência com PR Scanning, DeepScan e Code Security Intelligence. O processo humano ainda define impacto de negócio e aceita ou rejeita risco.

Caso de uso

Um assistente lê tickets não confiáveis e pode chamar uma ferramenta que modifica permissões. O risco surge quando o conteúdo do ticket influencia uma ação privilegiada sem autorização independente. O review deve procurar o local onde texto vira ação, quais argumentos são validados e quem autoriza a ferramenta.

Laboratório 09 - hipóteses e evidências

```
mkdir -p .evidence
cat > .evidence/threat-model.csv <<'CSV'
id,ativo,entrada,ameaca,controle,evidencia
TM01,produtos,query q,SQL injection,parametro SQL,preencher scan
TM02,faturas,id da URL,acesso entre usuarios,ownership server-side,nao implementado
TM03,ferramenta LLM,ticket externo,acao indevida,autorizacao independente,nao implementado
CSV
python3 - <<'PY'
import csv
with open('.evidence/threat-model.csv') as f:
    for row in csv.DictReader(f):
        print(row['id'], row['ameaca'], '=>', row['evidencia'])
PY
```

Associe TM01 ao módulo 3 e ao CWE de SQL injection da matriz. Para TM02/TM03, registre que são hipóteses de sistemas ainda não implementados no repo, não findings existentes. Abra a matriz fornecida e selecione outras três classes relevantes ao seu projeto real; acrescente ativo e controle específico a cada uma.

Critério: distinguir ameaça, finding confirmado e teste ainda não realizado. Diagnóstico: não converta toda linha da matriz em ticket sem um caminho de abuso; não declare uma hipótese resolvida apenas por scan vazio. Limpeza: mantenha o modelo local para o exercício de CSI.

Fonte: Vulnerability Coverage Matrix, páginas 1-5.

PLATFORM

10. Reporting

Mecanismo e decisões técnicas

Reporting apresenta atividade e resultados em janelas como 24 horas, 7, 30 e 90 dias. As métricas incluem PRs analisadas, findings, PRs mescladas e risco levado ao merge. O indicador de melhoria acompanha PRs nas quais findings deixam de aparecer em scans seguintes; não representa uma auditoria manual de cada correção.

As unidades importam: quantidade de findings, de scans e de PRs são números diferentes. Uma PR pode ter vários scans e vários findings. Uma queda no número de achados pode resultar de menos atividade, mudança de escopo, ajuste de configuração ou correção real. A interpretação deve acompanhar o volume de PRs, a janela e a composição de repositórios.

A API de findings é útil para investigação e exportação, mas uma consulta a all_findings não reproduz automaticamente todos os agregados do dashboard. Filtros de branch, execuções mais recentes e estado alteram o conjunto retornado. Relatórios locais deste curso devem ser identificados como recortes próprios.

Caso de uso

Um gestor quer avaliar se o processo evita risco antes do merge. Compare PRs com findings corrigidos, PRs que foram mescladas com risco e motivos de exceção; não use apenas a contagem absoluta de vulnerabilidades para comparar equipes com volumes diferentes.

Laboratório 10 - relatório com denominador

Na interface Reporting, selecione 7 dias e o escopo disponível. Registre os valores exibidos e a data/hora. Depois faça um recorte local de findings do repo de laboratório. O parâmetro days é obrigatório em all_findings; per_page limita a página, não o total.

```
mkdir -p .evidence
dryget "accounts/$AID/all_findings?days=7&repository_id=$RID&per_page=100&page=1" \
  > .evidence/findings-page1.json
jq 'keys' .evidence/findings-page1.json
jq . .evidence/findings-page1.json
cat > .evidence/reporting.md <<'TXT'
Janela e fuso:
Escopo de repositorios:
PRs analisadas:
Findings exibidos:
PRs mescladas com risco:
Mudancas de configuracao na janela:
Interpretacao e limites:
TXT
```

Inspecione os metadados de paginação retornados antes de contar resultados. Se houver páginas adicionais, repita com page=2, 3 etc., sem sobrescrever arquivos. Não some objetos de metadados como se fossem findings. Critério: relatório com unidade, janela, fonte e limitação explícitas. Diagnóstico: dados ausentes podem ser atividade fora da janela, conta incorreta ou filtros distintos. Limpeza: preserve apenas evidências necessárias em armazenamento privado.

Fonte: Reporting, páginas 1-4; referência API all_findings consultada em 20/10/2026.

PLATFORM

11. Risk Register

Mecanismo e decisões técnicas

Risk Register é o local de trabalho para investigar e acompanhar findings. Ele reúne o que foi detectado em diferentes superfícies de análise e permite usar filtros e estados para organizar tratamento. A decisão deve preservar a explicação técnica: o status sozinho não informa por que um finding foi considerado falso positivo ou aceito.

Separe correção de triagem. Resolved indica uma decisão/estado de tratamento; um novo scan é a evidência necessária para demonstrar que o caminho vulnerável desapareceu. Accepted Risk registra uma aceitação e não equivale a falha inexistente. False Positive significa que o raciocínio não se aplica ao código naquele contexto. Won't Fix/Nitpick comunica que não haverá correção, com justificativa apropriada.

O estado afeta workflows de formas diferentes. Na documentação de blocking, apenas False Positive e Won't Fix/Nitpick liberam automaticamente um check de falha relacionado à triagem. Accepted Risk, In Progress e Resolved não o liberam. Se sua política exige uma exceção formal de merge, ela deve ser tratada no processo de governança, não inferida pelo nome do status.

Caso de uso

Um finding SQLi real é atribuído para correção. Marcá-lo como In Progress ajuda o acompanhamento, mas o check continua falhando até que a condição de bloqueio seja removida. Uma discussão sobre prazo não muda o comportamento técnico do código.

Laboratório 11 - ficha de triagem

Crie uma nova ocorrência para não depender de uma PR já corrigida. Não envie feedback de falso positivo para a SQLi real do exercício.

```
lab_branch lab/11-triage
python3 "$KIT/change.py" unsafe
git add app.py
git commit -m 'Lab: finding para triagem'
lab_pr 'Lab 11 - Risk Register'
lab_evidence
cat > .evidence/triage.md <<'TXT'
Finding ID e scan:
Arquivo e linhas:
Entrada controlada:
Operacao sensivel:
Impacto demonstrado:
Estado escolhido: In Progress
Motivo: consulta vulneravel confirmada pelo teste
Plano: parametrizar e reexecutar testes/scan
TXT
```

No Risk Register, filtre pelo repositório/PR, abra o detalhe e registre o finding ID. Marque In Progress com justificativa. Confira que isso não equivale a correção. Para concluir:

```
python3 "$KIT/change.py" safe
python3 -m unittest -v
git add app.py
git commit -m 'Corrige finding em tratamento'
git push
lab_evidence
lab_close
```

Critério: histórico de decisão e evidência de correção no SHA novo. Diagnóstico: se o finding não aparecer, confira tipo de scan, filtros, status e permissões. Limpeza: feche a PR; evite manter triagens experimentais sem explicação.

Fonte: Risk Register, páginas 1-4; PR Blocking, páginas 1-6.

PLATFORM

12. Code Security Intelligence

Mecanismo e decisões técnicas

Code Security Intelligence permite investigar o código e o histórico com perguntas em linguagem natural. A documentação descreve workflows como Feature Ships, Resolved Risk, Vulnerability Trends, Architecture Risks, Developer Trends, Shadow AI, Incident Response, Application Summary, Security Reviews, New Feature Review e PR Variant Analysis.

A qualidade de uma pergunta depende do escopo: organização, repo, branch, janela e objetivo. Peça que cada conclusão cite arquivo/linha, PR, commit ou finding quando disponível. Se a ferramenta não tiver dados suficientes, a resposta deve separar observação de hipótese. Um texto fluente não substitui evidência.

Para modelagem de ameaça, CSI ajuda a levantar pontos de entrada, ativos, fluxos de dados e fronteiras de confiança. O analista revisa essa proposta e decide quais hipóteses exigem testes. O workflow de variantes pode procurar padrões relacionados a um bug; não deve declarar a erradicação de uma classe inteira com base em uma única correção.

Caso de uso

Após corrigir SQLi, o time quer saber se outras consultas usam concatenação e se uma biblioteca interna implementa parametrização. Uma consulta específica com evidências produz uma lista revisável para investigação; “o sistema está seguro?” tende a gerar uma resposta pouco acionável.

Laboratório 12 - consulta investigativa

No painel Code Security Intelligence, selecione o repo e execute o prompt abaixo. O comando cria um arquivo para reutilização no módulo MCP; não chama uma API de CSI fictícia.

```
cat > .evidence/csi-prompt.txt <<'TXT'
Analise somente o repositorio de treinamento selecionado.
Mapeie entradas HTTP, dados persistidos e operacoes SQL.
Para cada afirmacao, cite arquivo/linha e referencia analisada.
Explique a fronteira entre query string e comando SQL.
Separe fatos observados, hipoteses e dados ausentes.
Compare consulta parametrizada e concatenada, se ambas estiverem
no escopo consultavel. Nao suponha que branches fechadas foram lidas.
Nao modifique codigo, triagens ou configuracoes.
TXT
cat .evidence/csi-prompt.txt
```

Salve a consulta na interface quando essa opção estiver disponível. Compare a resposta com app.py e o modelo de ameaça do módulo 9. Escolha um workflow de Application Summary para visão inicial e um de Security Reviews/PR Variant Analysis para uma hipótese específica.

```
nl -ba app.py
cat .evidence/threat-model.csv
```

Critério: pelo menos uma afirmação verificada manualmente e uma limitação registrada. Diagnóstico: resposta genérica pode significar escopo incorreto, falta de dados/indexação ou pergunta vaga. Limpeza: mantenha a consulta salva com nome que identifique o repo e a finalidade.

Fonte: Code Security Intelligence, páginas 1-4.

PLATFORM

13. PR Scanning Configuration

Mecanismo e decisões técnicas

A Default Configuration governa repositórios sem uma configuração salva específica. Ao atribuir um repo a uma Saved Configuration, ele passa a ser governado exclusivamente por ela. Um repo pertence a uma configuração por vez. Não trate isso como duas regras independentes que se somam em execução.

As abas separam comentários, blocking, notificações, contexto, políticas e agentes. Por padrão documentado, os agentes estão habilitados, blocking está desligado, o limiar é High, comentários estão ligados e notificações desligadas. Salvar a configuração passa a afetar o próximo scan; não reinterpreta automaticamente todo o histórico.

Silent mode oculta a saída do agente/política no comentário, mas mantém análise, registro e eventual check de falha. Desligar Issue Comments também não desliga o scanner. Blocking geral usa severidade dos Code Security Agents; blocking por agente ou política é binário: qualquer finding daquela origem pode falhar independentemente da severidade.

Deduplicate Notifications compara a maior severidade do scan atual com a do anterior da mesma PR. Ela não faz deduplicação por finding ID. Um novo finding High após um High pode não gerar novo alerta; uma elevação de High para Critical pode gerar. A configuração aceita até sete políticas.

Caso de uso

Um time quer testar um agente sem comentários no GitHub. Silent mode permite observar os findings no Risk Register. Se Blocking permanecer ativo, o time poderá receber check de falha mesmo sem explicação no comentário; isso deve ser intencional e comunicado.

Laboratório 13 - configuração exclusiva e novo scan

Em Settings > Configuration, crie “Training-observe”, atribua somente o repo de laboratório e registre os valores iniciais. Ative comentários e confirmação de scan limpo; deixe blocking desligado e notificações desligadas. Salve. Não altere a Default Configuration da organização para este exercício.

```
lab_branch lab/13-config
printf '\nTeste de configuracao observavel.\n' >> README.md
git add README.md
git commit -m 'Lab: aplica configuracao no proximo scan'
lab_pr 'Lab 13 - configuracao de PR'
lab_evidence
# Consulta de inventario das configuracoes, se o token permitir:
dryget "accounts/$AID/configurations" \
  > .evidence/configurations.json
jq . .evidence/configurations.json
```

Desative temporariamente comentários na configuração de treinamento e faça um novo commit que altere app.py de forma segura. Observe checks/painel. Reative comentários ao final.

```
printf '\n# Laboratorio de observabilidade de scans.\n' >> app.py
git add app.py
git commit -m 'Lab: confirma scan com comentarios desativados'
git push
lab_evidence
lab_close
```

Critério: relacionar comportamento ao scan posterior ao Save. Diagnóstico: conferir atribuição exclusiva do repo, comentário de um scan anterior, agente silencioso e estado draft. Limpeza: reative comentários, mantenha blocking off e reserve a configuração para os próximos módulos.

Fonte: PR Scanning Configuration, páginas 1-10.

PLATFORM

14. Custom Code Policies

Mecanismo e decisões técnicas

Custom Code Policies expressam regras específicas em linguagem natural. Diferentemente dos agentes gerais, elas podem representar restrições de arquitetura ou de engenharia da organização. Uma boa política define comportamento proibido/obrigatório, escopo, exceções e exemplos suficientes para reduzir interpretações amplas.

Cada política pode receber severidade uniforme, Silent mode e Blocking ao ser anexada à configuração. Blocking de política é binário; um finding Low pode falhar se essa política estiver configurada para bloquear. O Background da política deve conter o contexto necessário: políticas não herdam automaticamente Repository Context nem AGENTS.md.

Evite instruções vagas como “siga as melhores práticas”. Prefira uma condição observável, por exemplo, não usar subprocess com shell=True quando o comando contém entrada de requisição. Uma regra assim exige investigar a origem da entrada, não proibir toda chamada a subprocess em scripts administrativos controlados.

Caso de uso

A organização proíbe eval em código Python de aplicação, inclusive quando um desenvolvedor acredita que os dados são internos. A política torna essa convenção explícita e repetível em PRs, sem esperar que uma regra genérica coincida com a decisão local.

Laboratório 14 - política com teste positivo e negativo

Na página Custom Code Policies, crie a regra usando os campos disponíveis. Texto da regra: “Report any call to Python eval() in application source files. Ignore documentation and test fixtures. Recommend explicit parsing or an allowlisted dispatch table.” No Background, informe que app.py é código de aplicação e que os arquivos test_* são testes. Anexe a Training-observe; severidade Medium, Blocking off e Silent mode off.

```
lab_branch lab/14-policy
cat > calculator.py <<'PY'
def calculate(user_expression):
    return eval(user_expression)
PY
git add calculator.py
git commit -m 'Lab: violacao da politica de eval'
lab_pr 'Lab 14 - custom code policy'
lab_evidence
```

Observe o finding da política e diferencie-o de eventual finding de agente geral. Substitua a interpretação arbitrária por uma operação explícita:

```
cat > calculator.py <<'PY'
def calculate_sum(left, right):
    return float(left) + float(right)
PY
python3 -m py_compile calculator.py
git add calculator.py
git commit -m 'Lab: substitui eval por operacao explicita'
git push
lab_evidence
lab_close
```

Critério: regra, escopo, exemplo positivo, exemplo negativo e resultados registrados. Diagnóstico: política não anexada, limite de sete, Silent mode ou exceção ambígua. Limpeza: remova a associação da política experimental se não quiser que ela afete módulos seguintes; mantenha um registro do texto usado.

Fonte: Custom Code Policies, páginas 1-5; PR Scanning Configuration, páginas 7-8.

PLATFORM

15. Repository Context

Mecanismo e decisões técnicas

Repository Context fornece informações que não são facilmente inferidas do código: autenticação compartilhada, contratos de bibliotecas internas, arquitetura, modelo de tenancy e controles externos. O contexto deve descrever fatos verificáveis. Dizer “todas as entradas são seguras” sem apontar onde e como são validadas pode esconder uma falha real.

Em PRs, pode-se associar documentos pela configuração e usar AGENTS.md na raiz do repositório. DeepScan também descobre AGENTS.md em subdiretórios. Se houver contexto específico configurado para DeepScan, ele substitui os AGENTS.md naquela análise. O alcance dos arquivos e essa precedência devem ser conhecidos para evitar resultados aparentemente contraditórios.

Contexto não é feedback de um finding individual. Use contexto para invariantes da arquitetura e triagem para a decisão sobre uma ocorrência concreta. Custom Code Policies precisam de seu próprio Background. Além disso, AGENTS.md pode ser lido por outros agentes de desenvolvimento: seu texto deve ser correto e apropriado para esses consumidores também.

Caso de uso

Uma biblioteca interna faz autorização por tenant antes de chamar handlers. Documentar o wrapper e o ponto onde a identidade é validada permite que a investigação verifique o controle. Uma alegação genérica de que “a aplicação fica atrás de um gateway” não prova autorização por objeto.

Laboratório 15 - contexto verificável

```
lab_branch lab/15-context
cat > AGENTS.md <<'TXT'
# Security context for the training repository
This is a local educational application, not a production service.
app.py binds to loopback and exposes GET /search?q=.
There is no authentication or tenant authorization middleware.
The SQLite database is created in memory with synthetic products.
SQL parameters must remain bound values, never SQL fragments.
Do not assume a gateway validates user input.
TXT
git add AGENTS.md
git commit -m 'Documenta contexto verificavel do repositorio'
lab_pr 'Lab 15 - repository context'
lab_evidence
```

No painel Repository Context, crie um documento equivalente e associe-o apenas à configuração de treinamento. Compare o texto com app.py, linha por linha. Para testar DeepScan, selecione essa branch e registre se há contexto específico configurado; não atribua uma diferença de resultado ao modelo sem verificar a precedência.

```
nl -ba AGENTS.md
nl -ba app.py
lab_close
```

Critério: cada afirmação de segurança aponta para evidência ou é explicitamente uma condição do laboratório. Diagnóstico: contexto não associado, branch diferente ou Background ausente na política. Limpeza: desassocie o documento experimental ou mantenha somente informações válidas para main.

Fonte: Repository Context, páginas 1-4.

PLATFORM

16. Finding Tuning with Feedback

Mecanismo e decisões técnicas

A documentação descreve aprendizado a partir de triagens False Positive, Won't Fix e Accepted Risk quando acompanhadas de explicação útil. Uma resposta vazia ou “noted” não oferece um precedente aplicável. Para findings de PR, o aprendizado ocorre depois do merge; para DeepScan, é imediato. In Progress e Resolved não são categorias de aprendizado.

Isso é diferente de liberar um check. Accepted Risk pode fornecer contexto para aprendizado, mas não libera automaticamente a falha de PR Blocking. False Positive e Won't Fix/Nitpick têm esse efeito de liberação documentado. O texto da decisão precisa deixar claro qual situação foi aceita e quais controles existem.

Um bom feedback descreve por que o fluxo é seguro ou por que o risco é aceito, aponta arquivos/controles e limita a generalização. Um feedback ruim afirma “falso positivo” porque a correção é inconveniente. Não use uma falha real de laboratório para ensinar ao scanner que concatenação SQL é segura.

Caso de uso

Um wrapper interno faz validação e autorização antes de uma operação marcada como suspeita. O analista confirma a implementação e fornece a referência concreta. Se a informação se aplica a vários achados e repositórios, complementa Repository Context em vez de repetir uma justificativa vaga em cada triagem.

Laboratório 16 - preparar feedback sem contaminar aprendizado

Escolha um finding real já investigado no ambiente autorizado. Se não houver falso positivo confirmado, execute a parte documental e use In Progress para uma falha real. Não invente uma conclusão para completar o exercício.

```
cat > .evidence/feedback-draft.txt <<'TXT'
Categoria proposta: preencher apos investigacao
Finding ID: preencher
Hipotese do scanner:
Controle efetivamente verificado:
Arquivo/linha e commit do controle:
Por que o controle cobre esta entrada e operacao:
Limites: quando esta justificativa NAO se aplica:
TXT
cat .evidence/feedback-draft.txt
```

O formato documentado de comentário permite @dryrunsecurity FP seguido do ID e da explicação. Só envie o comando abaixo depois de substituir a amostra por evidência real e selecionar uma PR correspondente. Won't Fix usa NIT no fluxo documentado; confirme a categoria e o texto antes de publicar.

```
# Exemplo de formato, nao envie os placeholders literalmente:
# @dryrunsecurity FP FINDING_ID explicacao tecnica verificavel
# Salve o comentario final em .evidence/feedback-final.txt.
# Publicacao opcional, somente para uma triagem verdadeira:
# gh pr comment NUMERO_DA_PR --body-file .evidence/feedback-final.txt
```

Critério: explicar categoria, efeito sobre check e momento de aprendizado separadamente. Para comprovar aprendizado em PR, use uma correção segura e um merge aprovado no processo da equipe; não mescle uma vulnerabilidade só para forçar o evento. Diagnóstico: PR não mesclada, justificativa insuficiente, categoria sem aprendizado ou contexto diferente. Limpeza: remova triagem incorreta no painel, documentando a correção da decisão.

Fonte: Finding Tuning with Feedback, páginas 1-6; PR Blocking, páginas 1-6.

PLATFORM

17. PR Blocking

Mecanismo e decisões técnicas

Bloqueio efetivo combina configuração DryRun e enforcement do SCM. DryRun envia check de falha quando a regra é satisfeita; o GitHub deve exigir esse check para impedir merge. Sem essa segunda parte, o check vermelho pode ser apenas informativo. Uma configuração do DryRun não cria automaticamente todas as proteções necessárias no GitHub.

Há blocking geral por severidade, blocking por agente e blocking por política. Os dois últimos são binários. No fluxo de rulesets de organização, a documentação orienta o check DryRun com origem DryRunSecurity. Configurações clássicas podem apresentar checks específicos de agentes/políticas. Descubra os nomes e app IDs efetivos na sua PR antes de criar uma regra.

GitHub Rulesets pode centralizar enforcement por organização. O laboratório abaixo usa proteção clássica de uma branch em um repo dedicado para tornar o payload revisável. A disponibilidade depende do plano e das permissões do GitHub. No GitLab, a opção Pipelines must succeed afeta o resultado do pipeline, não somente DryRun.

Caso de uso

A equipe define que High/Critical impedem merge. Uma SQLi High gera falha, mas o usuário ainda consegue mesclar porque main não exige o check. O problema está no enforcement, não na análise da vulnerabilidade.

Laboratório 17 - check obrigatório no repo dedicado

Na Training-observe, ligue PR blocking com limiar High. Mantenha os agentes/políticas sem bloqueios binários extras para isolar o teste. Salve. Abra uma PR vulnerável e aguarde sua análise.

```
lab_branch lab/17-blocking
python3 "$KIT/change.py" unsafe
git add app.py
git commit -m 'Lab: testa bloqueio de merge'
lab_pr 'Lab 17 - check obrigatorio'
SHA=$(gh pr view --json headRefOid --jq .headRefOid)
gh api "repos/$GH_REPO/commits/$SHA/check-runs" \
  --jq '.check_runs[] | {name, app_id:.app.id, conclusion}'
```

Somente no repo recém-criado para o curso, selecione o contexto e app ID reais do check DryRun. O PUT de proteção abaixo define a configuração completa representada no payload; não o aplique em uma branch de produção com regras preexistentes. Se já houver proteção, use a interface para editar a regra existente preservando os demais requisitos.

```
read -rp 'Nome exato do check: ' CHECK_NAME
read -rp 'App ID numerico do check: ' APP_ID
gh api "repos/$GH_REPO/branches/main/protection" \
  > .evidence/protection-before.json
# 404 e esperado se ainda nao houver protecao; outros erros exigem investigar.
jq -n --arg c "$CHECK_NAME" --argjson a "$APP_ID" '{
 required_status_checks:{strict:true,checks:[{context:$c,app_id:$a}]},
 enforce_admins:true,
 required_pull_request_reviews:null,
 restrictions:null
}' > .evidence/protection-lab.json
# Execute apenas depois de confirmar que nao existe protecao anterior.
gh api --method PUT "repos/$GH_REPO/branches/main/protection" \
  --input .evidence/protection-lab.json > .evidence/protection-after.json
```

Confira na interface que merge está impedido quando o check falha. Se o finding tiver severidade abaixo de High, não haverá bloqueio por esse limiar; registre o resultado ou ajuste a regra explicitamente para o objetivo do teste. Corrija e observe novamente:

```
python3 "$KIT/change.py" safe
python3 -m unittest -v
git add app.py
git commit -m 'Corrige condicao de bloqueio'
git push
gh pr checks --watch --interval 15
lab_evidence
lab_close
# Limpeza: apenas a protecao criada neste lab, se nao existia antes.
gh api --method DELETE "repos/$GH_REPO/branches/main/protection"
```

Desligue blocking na configuração de treinamento ao finalizar. Critério: falha impede merge e correção passa no mesmo check obrigatório. Diagnóstico: regra sem enforcement, app/context incorreto, bypass do ator, branch alvo diferente ou scan pendente. Nunca use aceitação de risco como promessa de liberação automática.

Fonte: PR Blocking, páginas 1-6; GitHub REST Branch protection.

PLATFORM

18. Permissions

Mecanismo e decisões técnicas

DryRun distingue Admin e Developer. A documentação associa administradores do GitHub ao papel Admin e membros ao papel Developer; no GitLab, Owner/Maintainer correspondem ao papel administrativo descrito. O acesso aos dados também depende do acesso aos repositórios no provedor.

Administradores podem gerir configurações, políticas e chaves e iniciar recursos administrativos como DeepScan e Insights. Desenvolvedores consultam os repositórios autorizados e podem participar da triagem conforme a funcionalidade disponível. A autorização de um token não deve ser presumida apenas porque a pessoa consegue abrir o dashboard: o token tem seu próprio escopo.

O desenho operacional deve separar quem investiga findings, quem altera critérios de bloqueio e quem administra integrações. Isso evita que uma mudança de visibilidade ou de gate passe despercebida durante uma correção urgente. O laboratório não altera permissões de pessoas na organização.

Caso de uso

Uma pessoa consegue ver a PR no GitHub, mas uma chamada da API DryRun retorna 403. Investigue organização, instalação, papel, escopo da chave e repo solicitado. Trocar por uma chave de administrador pode mascarar o problema e ampliar acesso desnecessariamente.

Laboratório 18 - matriz de acesso

```
gh api "repos/$GH_REPO" --jq '{full_name, private, permissions}'
dryget accounts | jq .
cat > .evidence/access-matrix.csv <<'CSV'
acao,papel_necessario,observado,evidencia
ver findings,acesso ao repositorio,preencher,preencher
alterar configuracao,Admin,preencher,preencher
criar chave,Admin,preencher,preencher
iniciar DeepScan,Admin conforme documentacao,preencher,preencher
CSV
```

Confira seu papel na interface e complete a matriz. Se houver um segundo usuário de teste já aprovado, compare a visibilidade sem compartilhar tokens. Se não houver, registre a limitação e não simule um teste que não ocorreu.

```
# Inspecao de HTTP sem imprimir o token:
curl --silent --show-error --output .evidence/access-response.json \
  --write-out '%{http_code}\n' \
  -H "Authorization: Bearer $DRYRUN_API_KEY" \
  "$DRYRUN_BASE/accounts/$AID/repositories"
```

Critério: distinguir identidade do usuário, permissão do SCM e escopo do token. Diagnóstico: 401 sugere credencial ausente/inválida/expirada; 403 sugere autorização insuficiente; 404 pode representar recurso incorreto ou não visível. Limpeza: nenhuma permissão de usuário deve ter sido alterada neste exercício.

Fonte: Permissions, páginas 1-3; API Access Keys, páginas 1-2.

PLATFORM

19. MCP

Mecanismo e decisões técnicas

O servidor MCP do DryRun fornece ferramentas para um cliente de IA consultar o contexto de segurança. O endpoint documentado é https://insights-mcp.dryrun.security/api/insights/mcp e a autenticação utiliza Bearer token. O cliente precisa iniciar a sessão MCP e descobrir as ferramentas; fazer um GET comum com curl não substitui uma conversa MCP completa.

Separe os componentes: o cliente Codex interpreta o pedido; o servidor MCP expõe as operações e dados autorizados; o DryRun é a fonte de contexto; o repositório local é onde uma eventual correção será preparada. A chave determina o acesso. Nomes e schemas das ferramentas devem ser descobertos pelo cliente, não adivinhados em um script.

Os PDFs mostram formatos genéricos de configuração para clientes. No Codex, a CLI abaixo registra o servidor HTTP e referencia uma variável de ambiente. Isso evita gravar o token literal no arquivo de configuração. Execute o Codex em um terminal que tenha a variável exportada.

Caso de uso

Um analista pede ao Codex os findings do repo e a explicação do caminho vulnerável. O assistente consulta DryRun e relaciona as evidências com os arquivos locais. Uma etapa posterior, explicitamente solicitada, prepara a correção. O laboratório inicial permanece em leitura para validar a conexão.

Laboratório 19 - conectar e consultar

```
npm install -g @openai/codex
codex --version
codex login
codex login status
# A variavel DRYRUN_API_KEY ja deve estar exportada.
codex mcp add dryrun \
  --url https://insights-mcp.dryrun.security/api/insights/mcp \
  --bearer-token-env-var DRYRUN_API_KEY
codex mcp list
codex mcp get dryrun
codex --sandbox read-only
```

No cliente aberto, peça: “Descubra as ferramentas MCP do DryRun disponíveis nesta sessão. Consulte os findings do repositório GH_OWNER/dryrun-training na conta indicada. Cite IDs e fontes. Não modifique código nem triagem. Se faltar permissão ou uma ferramenta, informe a limitação.” Substitua o nome do repo pelo real; o shell não expande variáveis no texto digitado manualmente no chat.

Critério: uma chamada de ferramenta bem-sucedida e uma resposta ligada ao repo correto. Listar uma configuração não comprova conexão funcional. Diagnóstico: variável não herdada pelo processo, chave sem escopo, URL incorreta, proxy ou credencial revogada. Não use codex mcp login como substituto automático para um Bearer token: esse fluxo é destinado a OAuth quando suportado pelo servidor.

```
# Limpeza opcional ao encerrar todo o treinamento:
# codex mcp remove dryrun
```

Fonte: MCP, páginas 1-4; comandos Codex conferidos na referência oficial Developer commands.

PLATFORM

20. DryRun API

Mecanismo e decisões técnicas

A Simple API usa a base https://simple-api.dryrun.security/v1. A maioria dos recursos fica sob account_id e, quando necessário, repository_id. O fluxo de descoberta é conta -> repositório -> execução ou finding. Os IDs são valores retornados pela API; não substitua um UUID por org/repo ou por número da PR.

O endpoint repositories/{RID}/findings lista findings de PR. Para combinar PR, DeepScan, SCA e políticas, use all_findings e os filtros correspondentes. O parâmetro days aceita a janela documentada de 1 a 365 dias. A consulta pode retornar apenas a página solicitada e, conforme os filtros/defaults, somente os resultados mais recentes; registre esses critérios ao exportar.

Requisições de leitura podem ser repetidas para diagnóstico. POST de DeepScan inicia trabalho e retorna aceitação, não conclusão. Se uma chamada de escrita perder a resposta, confira o estado antes de repetir para não gerar scans duplicados. Um HTTP 202 não comprova que todas as etapas concluíram.

Os PDFs listam endpoints de configuração e triagem, mas não imprimem todos os schemas expandidos. Para alterações desses recursos, consulte o Swagger/OpenAPI da versão atual e use a interface quando o contrato não estiver claro. O curso não inventa campos de payload para preencher essa lacuna.

Caso de uso

Você precisa exportar os findings de um repo e correlacioná-los com uma execução para criar tickets. Consulta a lista, guarda IDs e detalhes e só então transforma o resultado. Se o integrador tratar todo retorno como array simples, pode perder metadados de paginação ou falhar ao receber um erro JSON.

Laboratório 20 - descoberta, consulta e execução

```
dryget accounts | jq '.data[] | {account_id,org_name}'
dryget "accounts/$AID/repositories" | jq '.data[] | {id,name}'
dryget "accounts/$AID/repositories/$RID/scans" \
  > .evidence/pr-scans.json
dryget "accounts/$AID/repositories/$RID/findings" \
  > .evidence/pr-findings.json
dryget "accounts/$AID/all_findings?days=30&repository_id=$RID&finding_type=all&per_page=100&page=1" \
  > .evidence/all-findings.json
jq 'keys' .evidence/all-findings.json
# Selecione um ID real da resposta antes de consultar triagem.
read -rp 'Finding ID: ' FID
dryget "accounts/$AID/findings/$FID/triage" | jq .
```

Para iniciar um DeepScan pelo endpoint documentado, use o comando abaixo. Não acrescentamos um campo branch que não consta do exemplo de contrato usado. Para escolher outra branch, utilize o fluxo de interface do módulo 5.

```
curl --fail-with-body --silent --show-error \
  --request POST \
  -H "Authorization: Bearer $DRYRUN_API_KEY" \
  "$DRYRUN_BASE/accounts/$AID/repositories/$RID/deepscans" \
  | tee .evidence/deepscan-start.json
# Confira o estado na lista/painel antes de pedir resultados.
dryget "accounts/$AID/repositories/$RID/deepscans" | jq .
```

Inspecione também o contrato atual e os recursos de configuração sem alterá-los:

```
curl --fail --silent --show-error \
  https://simple-api.dryrun.security/api-docs/v1/swagger.yaml \
  -o .evidence/dryrun-openapi.yaml
dryget "accounts/$AID/analyzers" > .evidence/analyzers.json
dryget "accounts/$AID/custom_policies" > .evidence/policies.json
dryget "accounts/$AID/insights" > .evidence/insights.json
```

Critério: exportação identifica conta, repo, janela, página, tipo e execução. Diagnóstico: 400/422 exige revisar parâmetros/schema; 401 credencial; 403 escopo; 404 IDs; 429 limites e Retry-After; 5xx falha temporária a investigar com request_id se presente. Limpeza: evite retries automáticos de POST e não compartilhe respostas sensíveis fora do escopo do curso.

Fonte: DryRun API, páginas 1-6; Swagger UI/OpenAPI oficial.

API INTEGRATIONS

21. Webhooks

Mecanismo e decisões técnicas

Webhooks transportam notificações de findings de PR para um endpoint. Crie o destino nas configurações de Webhooks e depois selecione-o na configuração de PR, habilitando notificações e limiar. Criar um endpoint sem associá-lo ao fluxo não garante eventos. A documentação também descreve Slack e daily digest; o digest tem finalidade diferente do evento individual.

O payload genérico contém metadados do repo/PR e uma lista results. Cada item pode conter id, type, filename, line_start, line_end, description, risk e finding_type. Consuma o evento real recebido antes de fechar um contrato de integração. O enum de um campo não é garantia de que todos os tipos de scan emitam esse webhook: a documentação de entrega descreve PRs, não um stream completo de DeepScan/SCA.

O receiver do curso captura o JSON e não cria tickets automaticamente. Um hash do conteúdo evita salvar duas cópias de payloads idênticos. Isso não é deduplicação por finding, pois eventos com timestamps diferentes produzem hashes diferentes. Em produção, use fila, persistência, autenticação compatível com o emissor, observabilidade, retries e identidade de finding para idempotência.

Caso de uso

A equipe precisa validar a forma real da notificação antes de implementar um fluxo Jira. Capturar um evento e compará-lo ao schema evita implementar o parser para um exemplo simplificado que não corresponde ao ambiente.

Laboratório 21 - receptor e entrega de PR

Em um terminal Bash no repo, carregue as variáveis do curso. Gere um segredo de caminho aleatório. O script é integralmente transcrito no apêndice.

```
export WEBHOOK_PATH_KEY=$(python3 -c 'import secrets; print(secrets.token_urlsafe(32))')
printf 'Caminho temporario: /hook/%s\n' "$WEBHOOK_PATH_KEY"
python3 "$KIT/receiver.py"
```

Deixe esse terminal aberto. Em outro, faça um teste local com a amostra do kit. Copie o valor da variável do primeiro terminal sem publicá-lo no repositório.

```
read -rsp 'Mesmo segredo de caminho: ' WEBHOOK_PATH_KEY; echo
export WEBHOOK_PATH_KEY
curl --fail-with-body -X POST \
  -H 'Content-Type: application/json' \
  --data-binary "@$KIT/event-example.json" \
  "http://127.0.0.1:8080/hook/$WEBHOOK_PATH_KEY"
```

Para o DryRun alcançar o receptor, abra um túnel de desenvolvimento. Use apenas a opção de instalação do seu sistema abaixo; a opção Linux se aplica a Debian/Ubuntu com arquitetura suportada pelo pacote oficial. Se preferir, use um endpoint HTTPS de teste já disponibilizado pela sua organização.

```
# macOS; instalacao necessaria apenas uma vez
brew install cloudflared

# Alternativa Debian/Ubuntu; nao executar esta parte no macOS:
CF_ARCH=$(dpkg --print-architecture)
CF_RELEASE='https://github.com/cloudflare/cloudflared/releases/latest/download'
curl --fail --location \
  "$CF_RELEASE/cloudflared-linux-$CF_ARCH.deb" -o /tmp/cloudflared-lab.deb
sudo dpkg -i /tmp/cloudflared-lab.deb
```

```
# Nos dois sistemas, inicie o tunel:
cloudflared tunnel --url http://localhost:8080
```

A URL temporária será exibida pelo cloudflared. No DryRun, configure a URL completa HTTPS seguida de /hook/SEGREDO. Use Test, se disponível, e depois associe o webhook a Training-observe, habilite notificações e selecione um limiar compatível. Desligue deduplicação para observar cada scan deste exercício. Gere uma PR de SQLi como no módulo 3 com branch lab/21-webhook.

```
lab_branch lab/21-webhook
python3 "$KIT/change.py" unsafe
git add app.py
git commit -m 'Lab: evento para webhook'
lab_pr 'Lab 21 - webhook real'
# Apos a entrega, em outro terminal no repo:
ls -lt .evidence/events
# Selecione um arquivo real, nao suponha que o primeiro e o evento desejado.
jq . .evidence/events/ARQUIVO_REAL.json
lab_close
```

Critério: um evento de teste e um evento real distinguíveis. Diagnóstico: URL expirada, caminho errado, serviço parado, limiar, deduplicação, configuração não associada ou payload de teste sem results. Limpeza: desligue notificações/remova o destino de treinamento antes de encerrar receptor e túnel com Ctrl+C. O túnel é temporário e não oferece arquitetura de produção.

Fonte: Webhooks, páginas 1-5; PR Scanning Configuration, páginas 4-6; Cloudflare Quick Tunnels.

API INTEGRATIONS

22. Jira Integration

Mecanismo e decisões técnicas

A documentação propõe integração por intermediários como Tines ou Zapier, em vez de pressupor um conector nativo completo. O fluxo é receber evento, normalizar campos, filtrar risco, procurar ticket existente, criar/atualizar e registrar o resultado. A automação não deve considerar cada notificação como um novo finding.

Este laboratório implementa a parte central com Python e curl, como alternativa didática ao intermediário. Usa o payload genérico results do módulo 21. Os exemplos de eventos de páginas diferentes podem ter formatos distintos; não misture um objeto finding isolado com um envelope results sem normalização.

Jira Cloud REST v3 usa Atlassian Document Format para description. Projeto e tipo de issue precisam existir e o usuário precisa poder criar aquele tipo. O exemplo usa token pessoal de API sem escopos específicos via domínio do site para um script local. Se sua política exigir token com escopos/gateway api.atlassian.com, adapte a base e escopos conforme a documentação Atlassian da sua conta.

A etiqueta do curso deriva de repo + finding ID. Isso ajuda a pesquisar duplicatas, mas uma sequência “pesquisar e criar” não é atômica: dois workers podem criar duas issues. Uma integração de produção precisa de persistência e controle de concorrência, além de tratamento de limites e falhas parciais.

Caso de uso

Um finding High reaparece após um novo commit. Em vez de abrir outro ticket, o fluxo encontra a issue pela identidade e atualiza a evidência. A prioridade Jira deve seguir um mapeamento aprovado; o laboratório não presume que um nome de prioridade exista em toda instância.

Laboratório 22 - criar uma issue a partir de um finding

Na sua conta Atlassian, gere um token de API para o laboratório. No Jira, escolha um projeto de teste. Preencha as variáveis sem colocar o token em arquivo versionado.

```
export JIRA_BASE='https://SEU-SITE.atlassian.net'
export JIRA_EMAIL='SEU-EMAIL'
export JIRA_PROJECT='CHAVE_DO_PROJETO'
read -rsp 'Jira API token: ' JIRA_API_TOKEN; echo
export JIRA_API_TOKEN
curl --fail-with-body --silent --show-error \
  -u "$JIRA_EMAIL:$JIRA_API_TOKEN" \
  "$JIRA_BASE/rest/api/3/myself" | jq '{accountId,displayName}'
curl --fail-with-body --silent --show-error \
  -u "$JIRA_EMAIL:$JIRA_API_TOKEN" \
  "$JIRA_BASE/rest/api/3/issue/createmeta/$JIRA_PROJECT/issuetypes" \
  | tee .evidence/jira-types.json | jq .
read -rp 'ID do tipo de issue permitido: ' JIRA_ISSUETYPE_ID
export JIRA_ISSUETYPE_ID
```

Consulte também os campos exigidos para esse tipo. Campos obrigatórios extras precisam ser incluídos no payload antes da criação; o script gera apenas os campos comuns.

```
curl --fail-with-body --silent --show-error \
  -u "$JIRA_EMAIL:$JIRA_API_TOKEN" \
  "$JIRA_BASE/rest/api/3/issue/createmeta/$JIRA_PROJECT/issuetypes/$JIRA_ISSUETYPE_ID" \
  > .evidence/jira-fields.json
# Comece pela amostra sintetica; depois use um evento real selecionado.
python3 "$KIT/jira_payload.py" "$KIT/event-example.json" 0
jq . .evidence/jira-create.json
LABEL=$(cat .evidence/jira-label.txt)
curl --fail-with-body --silent --show-error --get \
  -u "$JIRA_EMAIL:$JIRA_API_TOKEN" \
  --data-urlencode "jql=project = \"$JIRA_PROJECT\" AND labels = \"$LABEL\"" \
  --data-urlencode 'maxResults=20' \
  "$JIRA_BASE/rest/api/3/search/jql" \
  | tee .evidence/jira-search.json | jq .
```

Se a busca falhar, não a trate como “nenhuma issue”. Se houver correspondência, use a issue existente. Somente com busca válida sem correspondência e payload revisado, crie:

```
curl --fail-with-body --silent --show-error \
  -u "$JIRA_EMAIL:$JIRA_API_TOKEN" \
  -H 'Content-Type: application/json' \
  --data-binary @.evidence/jira-create.json \
  "$JIRA_BASE/rest/api/3/issue" \
  | tee .evidence/jira-created.json | jq .
ISSUE_KEY=$(jq -r '.key // empty' .evidence/jira-created.json)
test -n "$ISSUE_KEY" && curl --fail-with-body --silent --show-error \
  -u "$JIRA_EMAIL:$JIRA_API_TOKEN" \
  "$JIRA_BASE/rest/api/3/issue/$ISSUE_KEY?fields=summary,status,labels" | jq .
```

Para encerrar o ticket de teste, descubra uma transição permitida. IDs de transição variam por workflow; nunca use um número fixo sem consultar.

```
curl --fail-with-body --silent --show-error \
  -u "$JIRA_EMAIL:$JIRA_API_TOKEN" \
  "$JIRA_BASE/rest/api/3/issue/$ISSUE_KEY/transitions" | jq .
read -rp 'ID da transicao de encerramento: ' TRANSITION_ID
jq -n --arg id "$TRANSITION_ID" '{transition:{id:$id}}' \
  > .evidence/jira-transition.json
curl --fail-with-body --silent --show-error \
  -u "$JIRA_EMAIL:$JIRA_API_TOKEN" \
  -H 'Content-Type: application/json' \
  --data-binary @.evidence/jira-transition.json \
  "$JIRA_BASE/rest/api/3/issue/$ISSUE_KEY/transitions"
```

Critério: issue contém origem, localização, risco e link; nova busca pela label encontra a mesma identidade. Diagnóstico: 400 campos obrigatórios/ADF, 401 token/base, 403 permissão, 404 projeto/issue invisível, 429 rate limit. Limpeza: encerre o ticket de teste e revogue o token quando terminar. Para produção, revise a integração Tines/Zapier da documentação com os mesmos controles.

Fonte: Jira Integration, páginas 1-5; Atlassian Basic auth, Issues e Issue search REST v3.

API INTEGRATIONS

23. API Access Keys

Mecanismo e decisões técnicas

Chaves permitem acesso programático sem reutilizar a sessão interativa do dashboard. A criação exige escolher pelo menos um escopo. A chave pode dar acesso a mais de uma conta conforme sua autorização; por isso accounts é o primeiro passo de descoberta. O segredo completo é mostrado uma única vez.

Planeje a chave pelo consumidor: exportador somente leitura, integração de automação ou agente de remediação. Use os menores escopos que permitam a operação real e revise acesso quando o consumidor mudar. Os limites de uso devem ser consultados na conta; o curso não pressupõe um número fixo de requisições por minuto.

Rotação tem uma sequência: criar substituta, atualizar consumidor, verificar operação e revogar antiga. Revogar primeiro pode interromper a integração. Em um incidente de exposição, a prioridade pode ser revogar imediatamente e restaurar o serviço com uma chave nova conforme o processo de resposta.

Caso de uso

O token do receptor de automação foi usado também por uma estação de desenvolvimento. Separar chaves permite revogar a estação sem interromper o serviço e torna o propósito de cada credencial mais claro.

Laboratório 23 - leitura, substituição e revogação

Em Settings > Access Keys, crie “training-read” com escopos de leitura suficientes para accounts/repositories/findings. Confira as opções reais da interface; não suponha que o nome do escopo seja igual ao nome da rota. Copie o valor uma vez e use o terminal:

```
read -rsp 'Nova chave DryRun: ' DRYRUN_NEW_KEY; echo
curl --fail-with-body --silent --show-error \
  -H "Authorization: Bearer $DRYRUN_NEW_KEY" \
  "$DRYRUN_BASE/accounts" | jq .
# Depois da validacao, substitua a variavel do consumidor local.
export DRYRUN_API_KEY="$DRYRUN_NEW_KEY"
unset DRYRUN_NEW_KEY
```

Reinicie clientes que carregaram a variável antiga, incluindo a sessão Codex. Revogue a chave anterior no painel somente após identificar quais consumidores a utilizam. Teste a chave nova novamente. Para comprovar revogação da antiga, use uma variável temporária carregada pelo gerenciador de segredos e descarte-a em seguida; não imprima o valor.

```
read -rsp 'Chave antiga revogada para teste: ' REVOKED_KEY; echo
curl --silent --show-error --output /tmp/dryrun-revoked.json \
  --write-out '%{http_code}\n' \
  -H "Authorization: Bearer $REVOKED_KEY" \
  "$DRYRUN_BASE/accounts"
unset REVOKED_KEY
```

Critério: substituta funciona e antiga deixa de autorizar. Diagnóstico: token truncado, espaço copiado, escopo insuficiente ou processo com ambiente antigo. Limpeza: revogue apenas chaves de laboratório sem consumidores restantes; unset DRYRUN_API_KEY ao concluir o curso.

Fonte: API Access Keys, páginas 1-2.

API INTEGRATIONS

24. Remediation

Mecanismo e decisões técnicas

Remediação conecta o finding a uma ferramenta de programação por IA para preparar uma alteração contextualizada. A documentação cita Codex, Claude Code, Cursor, GitHub Copilot, Windsurf e VS Code. A integração combina chave, MCP e skill, mas a correção ainda precisa ser revisada e verificada.

O ciclo completo é: identificar finding e SHA, reproduzir a condição, preparar patch mínimo, executar teste que falhava, revisar diff, atualizar a PR e aguardar scan do novo commit. “O agente disse que corrigiu” não é critério de conclusão. Um patch pode remover o sintoma, quebrar compatibilidade ou deixar outro caminho vulnerável.

Em SCA, atualização exige avaliar compatibilidade e regenerar lockfile com o gerenciador do projeto. Em DeepScan, a correção deve partir do estado atual do código; a linha de um scan antigo pode ter mudado. Vincule o ticket e o finding ao novo patch sem assumir que posição de linha é uma identidade estável.

Caso de uso

Um finding indica concatenação SQL. O agente pode corrigir com binding de parâmetros e preservar a resposta da API. O teste de regressão verifica que o payload de injeção passa a ser dado literal e que a busca normal continua funcionando.

Laboratório 24 - correção assistida no Codex

Conclua o módulo 19 e instale a skill do módulo 25 antes de iniciar a sessão abaixo. A ordem de leitura segue o programa solicitado; a dependência de instalação é explícita.

```
lab_branch lab/24-remediation
python3 "$KIT/change.py" unsafe
git add app.py
git commit -m 'Lab: alvo de remediacao assistida'
lab_pr 'Lab 24 - remediacao com Codex'
lab_evidence
python3 -m unittest -v
# Falha do teste de injecao confirma a condicao inicial.
codex --sandbox workspace-write
```

No Codex, envie: “Use o finding DryRun desta PR e o código local. Confirme conta, repo, finding ID e SHA antes de editar. Corrija a SQL injection em app.py com a menor mudança que preserve o contrato de search. Execute os testes, explique o diff e não mescle a PR. Não altere configuração, severidade ou triagem para fazer o check passar.” Inclua o ID real ou cole o comentário do finding, conforme o workflow da skill.

Depois da edição, execute a verificação humana abaixo. Se o agente já tiver commitado/enviado, confira o histórico antes de criar outro commit vazio.

```
git status --short
git diff
git log -3 --oneline
python3 -m unittest -v
# Se as alteracoes ainda estiverem sem commit:
git add app.py
git commit -m 'Corrige SQLi com verificacao de regressao'
git push
lab_evidence
gh pr checks --watch --interval 15
```

Critério: teste antes falhava, agora passa; comportamento normal preservado; scan final corresponde ao SHA enviado. Se o agente modificou testes para aceitar a vulnerabilidade, rejeite o patch. Diagnóstico: ferramenta não acessível, finding antigo, branch errada ou correção parcial. Limpeza: lab_close; retenha diff e evidências de teste/scan.

Fonte: Remediation, páginas 1-2; DryRun Skill, páginas 1-3.

API INTEGRATIONS

25. DryRun Skill

Mecanismo e decisões técnicas

As skills adicionam instruções operacionais ao agente. A documentação descreve três skills: workflows de autoria/revisão de PR e remediação, incluindo dryrun-finding-remediation para DeepScan e SCA fora de PRs abertas. O instalador detecta o cliente e posiciona os arquivos correspondentes. Consulte Settings > Integrations para instruções específicas do seu agente.

No fluxo de autoria, o agente prepara uma branch e abre PR, permitindo o scan antes do merge. No fluxo de revisão, acompanha resultados após cada commit. Na remediação, usa o finding e seu contexto para propor uma correção. A skill de findings exige DRYRUN_API_KEY e permite consultar achados já presentes na base principal.

Há um detalhe operacional importante: o fluxo documentado fala em aguardar comentários, mas a configuração padrão pode não comentar scans limpos. O workflow deve conferir conclusão dos checks/scan e ausência de findings, sem esperar indefinidamente por um comentário que não será publicado. Uma branch com push direto para main não fornece o ciclo de PR desejado.

Caso de uso

Você pede ao agente uma pequena melhoria. A skill orienta a criação da PR, apresentação dos findings e uma nova checagem após correções. O ganho é incorporar a segurança ao processo de entrega; a skill não substitui permissões do GitHub nem enforcement de branch.

Laboratório 25 - instalar e exercitar o ciclo

Instale no ambiente em que o Codex será executado. O comando npx executa o instalador do projeto publicado; revise as opções apresentadas e selecione o cliente/escopo desejados.

```
npx skills@latest add DryRunSecurity/external-plugin-marketplace
# Reinicie a sessao do agente para carregar as skills instaladas.
test -n "$DRYRUN_API_KEY" && echo 'Variavel DryRun presente'
codex --sandbox workspace-write
```

No cliente, solicite: “Use as skills DryRun instaladas. Adicione documentação explicando a consulta parametrizada do repo. Trabalhe em uma branch lab/25-skill, abra PR e aguarde o scan do SHA final. Apresente findings ou evidência de scan limpo. Não faça merge.” O agente pode precisar de autorização do ambiente para usar rede/gh; siga o fluxo local de aprovação.

Após o agente atuar, confira:

```
git status --short
git branch --show-current
gh pr view --json number,url,headRefOid,state
lab_evidence
gh pr checks
```

Para o workflow DeepScan/SCA, use: “Use dryrun-finding-remediation para revisar os findings DeepScan deste repo. Liste candidatos com IDs e evidências; aguarde minha escolha antes de editar.” Selecione um finding verdadeiro, faça a correção em nova PR e siga a validação do módulo 24. Não crie uma vulnerabilidade artificial em main para exercitar esse caminho.

Critério: PR criada, revisão ligada ao SHA final e resultados apresentados sem espera infinita por comentário. Diagnóstico: skill não carregada no cliente, instalação em escopo diferente, variável ausente ou conta/repo incorretos. Limpeza: feche PRs de documentação de teste ou mescle apenas se desejado e aprovado; remova a conexão MCP/revogue tokens de laboratório ao encerrar.

Fonte: DryRun Skill, páginas 1-3; Remediation, páginas 1-2.

Exercício final - do risco à evidência

Execute um ciclo completo com a SQLi em uma branch nova lab/final. Use a configuração exclusiva, abra PR, identifique finding, escreva uma triagem In Progress, capture a notificação, gere um payload Jira e prepare a correção com Codex. Valide os testes locais e o scan do SHA final. Não faça merge da versão vulnerável.

Sua entrega deve conter: repo/PR/commit; finding ID; cadeia entrada -> operação sensível; resultado do teste antes/depois; decisão de triagem; IDs dos scans; configuração de bloqueio aplicada; ticket de teste, se Jira estiver disponível; limites da avaliação. A falta de Jira deve ser declarada como etapa não executada, não como sucesso presumido.

Avaliação sugerida: 20% reprodução e explicação da falha; 20% uso correto de configuração/escopo; 20% evidências de correção; 20% integração e tratamento de falhas; 20% clareza das limitações e rastreabilidade. Um scan vazio sem teste ou um patch sem novo scan não conclui o exercício.

```
# Inventario final de PRs de laboratorio ainda abertas:
gh pr list --state open --json number,title,headRefName,url
# Feche individualmente somente as PRs do treinamento:
# gh pr close NUMERO
# Confira estado local antes de encerrar:
git status --short
codex mcp list
# Limpeza opcional da integracao do curso:
# codex mcp remove dryrun
unset DRYRUN_API_KEY JIRA_API_TOKEN WEBHOOK_PATH_KEY
```

No painel, remova o webhook temporário, desassocie políticas/contextos de teste, revise a configuração exclusiva e revogue as chaves sem consumidores. Preserve o repo privado se precisar repetir o curso. Não há comando de exclusão automática de repositório neste material.

Referência rápida de diagnóstico

Scan e GitHub

PR sem comentário: confira draft/ready, Issue Comments, Show comment for no findings, Silent mode e conclusão dos checks. PR sem check: confira instalação da App, repo selecionado, evento/branch e logs do painel. PR pode mesclar com check vermelho: confira enforcement, check/app exigido, branch alvo e bypass. Finding persistente após patch: confira SHA analisado, término do scan e outros caminhos vulneráveis.

Erro src refspec main: confira branch local e existência de commit. Erro SSH no remote github-veracode: confira git remote -v, alias no SSH e qual conta tem acesso; o nome do remote e o nome do host SSH são coisas diferentes. Repositório existe mas gh não o acessa: confira gh auth status e GH_REPO. Não resolva um problema de identidade forçando push.

API, MCP e integração

401: token, validade e header. 403: escopo/papel/repo. 404: ID ou acesso. 429: limite/Retry-After. 5xx: guarde request_id se presente e evite repetir escrita às cegas. MCP configurado mas sem ferramenta: valide conexão real e ambiente do processo. Webhook sem entrega: destino associado, limiar, deduplicação e URL ativa. Jira 400: campos obrigatórios, tipo de issue e ADF. Busca Jira falhou: interrompa criação para evitar duplicatas.

Checklist de evidência

Cada conclusão deve informar o que foi observado, em qual referência, quando e com qual configuração. Separe teste local de scan remoto. Separe exemplo sintético de evento real. Separe uma hipótese de ameaça de um finding confirmado. Registre as etapas não executadas por falta de acesso e não as apresente como validação concluída.

Apêndice A - arquivos completos do kit

Os arquivos abaixo são auxiliares autorais do treinamento. Salve-os com os nomes indicados na pasta definida por KIT. Apenas app.py e test_app.py são copiados para o repositório inicial. O kit não contém credenciais nem acessa serviços por conta própria. A amostra de webhook é sintética e não reproduz obrigatoriamente todos os campos de um evento real.

O receptor não é um serviço de produção: opera com um processo, limite simples de tamanho e autenticação por caminho aleatório. O gerador Jira prepara um payload para revisão, sem fazer chamadas de rede. O segredo de caminho não é uma assinatura de webhook fornecida pelo DryRun.

lab.sh

```
# Funcoes didaticas deste curso; nao sao comandos oficiais DryRun.
# Execute em Bash: source /caminho/kit/lab.sh
export DRYRUN_BASE='https://simple-api.dryrun.security/v1'
lab_branch() {
  test -z "$(git status --porcelain)" || {
    echo 'Salve as alteracoes antes de trocar de branch.'; return 1;
  }
  git switch main && git pull --ff-only && git switch -c "$1"
}
lab_pr() {
  git push -u origin HEAD || return
  gh pr create --base main --title "$1" --body \
    'Laboratorio DryRun. Nao mesclar codigo vulneravel.'
}
lab_evidence() {
  local n sha
  n=$(gh pr view --json number --jq .number) || return
  sha=$(gh pr view --json headRefOid --jq .headRefOid) || return
  mkdir -p .evidence
  gh api "repos/$GH_REPO/commits/$sha/check-runs" \
    > ".evidence/pr-$n-checks.json"
  gh api --paginate "repos/$GH_REPO/issues/$n/comments" \
    > ".evidence/pr-$n-comments.json"
  gh pr checks || true
}
lab_close() {
  gh pr close || return
  git switch main
}
dryget() {
  test -n "$DRYRUN_API_KEY" || { echo 'Falta DRYRUN_API_KEY'; return 1; }
  curl --fail-with-body --silent --show-error \
    -H "Authorization: Bearer $DRYRUN_API_KEY" \
    "$DRYRUN_BASE/$1"
}
```

app.py

```
"""Aplicacao didatica: somente loopback; nenhuma autenticacao real."""
import json
import sqlite3
from http.server import BaseHTTPRequestHandler, HTTPServer
from urllib.parse import parse_qs, urlparse


def database():
    db = sqlite3.connect(':memory:')
    db.execute('CREATE TABLE products (name TEXT, price INTEGER)')
    db.executemany('INSERT INTO products VALUES (?, ?)',
                   [('book', 10), ('pencil', 2)])
    return db


def search(db, term):
    return db.execute(
        'SELECT name, price FROM products WHERE name = ?',
        (term,)).fetchall()


class Handler(BaseHTTPRequestHandler):
    def do_GET(self):
        parsed = urlparse(self.path)
        if parsed.path != '/search':
            self.send_error(404)
            return
        term = parse_qs(parsed.query).get('q', [''])[0]
        with database() as db:
            result = search(db, term)
        body = json.dumps(result).encode()
        self.send_response(200)
        self.send_header('Content-Type', 'application/json')
        self.end_headers()
        self.wfile.write(body)


if __name__ == '__main__':
    HTTPServer(('127.0.0.1', 8090), Handler).serve_forever()
```

test_app.py

```
import unittest
from app import database, search


class SearchTest(unittest.TestCase):
    def test_known_product(self):
        with database() as db:
            self.assertEqual(search(db, 'book'), [('book', 10)])

    def test_injection_is_literal(self):
        with database() as db:
            self.assertEqual(search(db, "' OR 1=1 --"), [])


if __name__ == '__main__':
    unittest.main()
```

change.py

```
"""Altera apenas app.py no repositorio didatico, com checagem de estado."""
import pathlib
import sys
p = pathlib.Path('app.py')
safe = """return db.execute(
        'SELECT name, price FROM products WHERE name = ?',
        (term,)).fetchall()"""
unsafe = """query = "SELECT name, price FROM products WHERE name = '" + term + "'"
    return db.execute(query).fetchall()"""
mode = sys.argv[1]
assert mode in ('unsafe', 'safe')
before, after = (safe, unsafe) if mode == 'unsafe' else (unsafe, safe)
text = p.read_text()
if text.count(before) != 1:
    raise SystemExit('Estado inesperado de app.py; revise o diff.')
p.write_text(text.replace(before, after))
```

receiver.py

```
"""Receptor de laboratorio. Captura JSON; nao encaminha automaticamente ao Jira."""
import hashlib
import hmac
import json
import os
from pathlib import Path
from http.server import BaseHTTPRequestHandler, HTTPServer

KEY = os.environ['WEBHOOK_PATH_KEY']
OUT = Path('.evidence/events')
OUT.mkdir(parents=True, exist_ok=True)


class Handler(BaseHTTPRequestHandler):
    def log_message(self, *args):
        pass  # Nao registrar a URL que contem o segredo do laboratorio.

    def do_POST(self):
        if not hmac.compare_digest(self.path, '/hook/' + KEY):
            self.send_error(404)
            return
        try:
            size = int(self.headers.get('Content-Length', '0'))
            if not 0 < size <= 1024 * 1024:
                self.send_error(413)
                return
            data = json.loads(self.rfile.read(size))
            if not isinstance(data, dict):
                raise ValueError('Objeto JSON esperado')
        except (ValueError, json.JSONDecodeError):
            self.send_error(400)
            return
        raw = json.dumps(data, sort_keys=True).encode()
        digest = hashlib.sha256(raw).hexdigest()
        dest = OUT / (digest + '.json')
        try:
            with dest.open('xb') as f:
                f.write(raw)
            print('Evento salvo:', dest, flush=True)
        except FileExistsError:
            print('Repeticao ignorada:', digest, flush=True)
        self.send_response(202)
        self.end_headers()
        self.wfile.write(b'{"accepted":true}')


if __name__ == '__main__':
    HTTPServer(('127.0.0.1', 8080), Handler).serve_forever()
```

jira_payload.py

```
"""Converte UM finding do webhook generico para Jira Cloud v3."""
import json
import os
import sys
from pathlib import Path

event = json.loads(Path(sys.argv[1]).read_text())
index = int(sys.argv[2]) if len(sys.argv) > 2 else 0
finding = event['results'][index]
repo = event.get('repo_full_name', 'repository-not-provided')
ident = str(finding['id'])
import hashlib
label = 'dryrun-' + hashlib.sha256(
    (repo + ':' + ident).encode()).hexdigest()[:24]
text = '\n'.join([
    'DryRun finding: ' + ident,
    'Repository: ' + repo,
    'PR: ' + str(event.get('pr_number', '')),
    'Risk: ' + str(finding.get('risk', '')),
    'File: ' + str(finding.get('filename', '')),
    'Line: ' + str(finding.get('line_start', '')),
    str(finding.get('description', '')),
    'Dashboard: ' + str(event.get('dashboard_url', '')),
])
payload = {'fields': {
    'project': {'key': os.environ['JIRA_PROJECT']},
    'issuetype': {'id': os.environ['JIRA_ISSUETYPE_ID']},
    'summary': ('[DryRun] ' + str(finding.get('type', 'Finding'))
                + ' - ' + repo)[:240],
    'labels': [label],
    'description': {'type': 'doc', 'version': 1, 'content': [
        {'type': 'paragraph', 'content': [{'type': 'text', 'text': text}]}]},
}}
Path('.evidence/jira-create.json').write_text(json.dumps(payload, indent=2))
Path('.evidence/jira-label.txt').write_text(label)
print('Payload pronto. Label:', label)
```

event-example.json

```
{
  "repo_full_name": "training/dryrun-training",
  "pr_number": 1,
  "dashboard_url": "https://app.dryrun.security",
  "results": [{
    "id": "lab-sqli-001", "type": "SQL Injection",
    "filename": "app.py", "line_start": 20, "line_end": 21,
    "description": "Exemplo sintetico: entrada concatenada na consulta.",
    "risk": "High", "finding_type": "pullrequest"
  }]
}
```

Apêndice B - fontes e rastreabilidade

A base principal são os 25 PDFs oficiais fornecidos pelo participante, impressos em 20/10/2026. As páginas citadas em cada módulo são as páginas desses arquivos, não as deste treinamento. O guia prático anterior também foi fornecido, mas não foi usado como fonte normativa. Este curso é uma elaboração didática, não uma tradução literal integral dos PDFs.

A documentação online foi usada para complementar os contratos de API e validar comandos externos em 20/10/2026. Versões de clientes, rótulos da interface, planos e permissões podem mudar; registre as versões do seu ambiente ao executar. Nas divergências entre exemplos de páginas, o laboratório preserva a distinção entre payload genérico, evento sintético e contrato efetivamente recebido.

API Access Keys - DryRun Security Docs.pdf - https://docs.dryrun.security/api-access-keys

Code Security Intelligence - DryRun Security Docs.pdf - https://docs.dryrun.security/code-security-intelligence

Custom Code Policies - DryRun Security Docs.pdf - https://docs.dryrun.security/custom-code-policies

Documentation - DryRun Security Docs.pdf - https://docs.dryrun.security/

DryRun API - DryRun Security Docs.pdf - https://docs.dryrun.security/dryrun-api

DryRun Skill - DryRun Security Docs.pdf - https://docs.dryrun.security/dryrun-skill

Finding Tuning with Feedback - DryRun Security Docs.pdf - https://docs.dryrun.security/finding-tuning

IaC Scanning - DryRun Security Docs.pdf - https://docs.dryrun.security/iac-scanning

Jira Integration - DryRun Security Docs.pdf - https://docs.dryrun.security/jira-integration

Language and Framework Support - DryRun Security Docs.pdf - https://docs.dryrun.security/language-support

MCP - DryRun Security Docs.pdf - https://docs.dryrun.security/mcp

PR Blocking - DryRun Security Docs.pdf - https://docs.dryrun.security/pr-blocking

PR Scanning - DryRun Security Docs.pdf - https://docs.dryrun.security/pr-scanning

PR Scanning Configuration - DryRun Security Docs.pdf - https://docs.dryrun.security/pr-scanning-configuration

Permissions - DryRun Security Docs.pdf - https://docs.dryrun.security/permissions

Quick Start - DryRun Security Docs.pdf - https://docs.dryrun.security/quick-start

Remediation - DryRun Security Docs.pdf - https://docs.dryrun.security/remediation

Reporting - DryRun Security Docs.pdf - https://docs.dryrun.security/reporting

Repository Context - DryRun Security Docs.pdf - https://docs.dryrun.security/repository-context

Repository Scanning with DeepScan - DryRun Security Docs.pdf - https://docs.dryrun.security/deepscan

Risk Register - DryRun Security Docs.pdf - https://docs.dryrun.security/risk-register

SCA - DryRun Security Docs.pdf - https://docs.dryrun.security/sca

Secrets Scanning - DryRun Security Docs.pdf - https://docs.dryrun.security/secrets-scanning

Vulnerability Coverage Matrix - DryRun Security Docs.pdf - https://docs.dryrun.security/vulnerability-coverage-matrix

Webhooks - DryRun Security Docs.pdf - https://docs.dryrun.security/webhooks

Referências externas verificadas

GitHub CLI - autenticação e repositórios: https://cli.github.com/manual/gh_auth_login e https://cli.github.com/manual/gh_repo_create

GitHub CLI - criação e checks de PR: https://cli.github.com/manual/gh_pr_create e https://cli.github.com/manual/gh_pr_checks

GitHub CLI - chamadas REST: https://cli.github.com/manual/gh_api

GitHub REST - proteção de branches: https://docs.github.com/en/rest/branches/branch-protection

OpenAI - instalação do Codex CLI: https://learn.chatgpt.com/docs/codex/cli

OpenAI - comandos do Codex, MCP e sandbox: https://learn.chatgpt.com/docs/developer-commands?surface=cli

Atlassian - autenticação do script local: https://developer.atlassian.com/cloud/jira/platform/basic-auth-for-rest-apis/

Atlassian - criação, metadados e transições: https://developer.atlassian.com/cloud/jira/platform/rest/v3/api-group-issues/

Atlassian - busca por JQL: https://developer.atlassian.com/cloud/jira/platform/rest/v3/api-group-issue-search/

Cloudflare - túnel temporário: https://developers.cloudflare.com/tunnel/get-started/quick-tunnels/

Cloudflare - pacote Linux: https://developers.cloudflare.com/tunnel/guides/update-cloudflared/

DryRun - contrato da Simple API: https://simple-api.dryrun.security/api-docs/index.html

DryRun - OpenAPI: https://simple-api.dryrun.security/api-docs/v1/swagger.yaml

Limites da validação deste material

Os scripts locais foram verificados no ambiente de autoria: aplicação segura, reprodução da falha SQLi, restauração da consulta parametrizada, sintaxe dos blocos Bash e geração de payload Jira. A execução nas suas contas DryRun, GitHub, Codex e Jira depende das credenciais, permissões e configurações reais; ela não foi realizada em seu nome durante a elaboração. Os laboratórios indicam os passos para você validar essas integrações.